Module 1
Foundations of Generative AI and Prompt Engineering

Module 2
Introduction to LangChain in GenAI Applications

LangChain Core Components and Advanced Features

- Recap: Introduction to LangChain
  Reading • 15 min
- LangChain: Core Concepts
  Video • 6 min
- LangChain Chains and Agents for Building Applications
  Video • 6 min
- (Optional) LangChain LCEL Chaining Method
  Video • 4 min
- Lab: Build Smarter AI Apps: Empower LLMs with LangChain
  Ungraded App Item • 1h
- Practice Quiz: LangChain Core Components and Advanced Features
  Practice Assignment • Grade: 100%

Module Summary and Evaluation

- Summary and Highlights: Introduction to LangChain in GenAI Applications
  Reading • 2 min
- Cheat Sheet: Introduction to LangChain in GenAI Applications
  Reading • 30 min
- Graded Quiz: Introduction to LangChain in GenAI applications
  Graded Assignment • Grade: 100%

Module 3
Build a Generative AI Application with LangChain

Application Development Workflow with Generative AI

- Choose the Right AI Model for Your Use Case
  Video • 4 min
- From Idea to AI: Building Applications with Generative AI
  Video • 7 min
- Introduction to Flask
  Video • 7 min
- (Optional) Reading: Flask: A Gateway to Web Development in Python
  Reading • 15 min
- (Optional) Reading: Python with Flask for Large Scale Projects
  Reading • 15 min
- **Cheat Sheet: Web Development Using Flask**
  Reading • 5 min
- Hands-on with GenAI: Choosing the Right Model for Your Application
  Ungraded App Item • 1h
- Practice Quiz: Application Development Workflow with Generative AI
  Practice Assignment • 10 min

Summary and Evaluation

- Summary and Highlights: Build a Generative AI Application with LangChain
  Reading • 3 min
- Cheat Sheet: Build GenAI Application with LangChain
  Reading • 30 min
- Graded Quiz: Build a Generative AI Application with LangChain
  Graded Assignment • 21 min

Course Wrap Up

- Course Wrap-up
  Video • 4 min
- Congratulations and Next Steps
  Reading • 2 min
- Team and Acknowledgments
  Reading • 2 min

Cheat Sheet: Web Development Using Flask

0:13 / 1:54 1x

Package/Method	Description	Code Example
Flask	Used to instantiate an object of the Flask class named app.	`from flask import Flask` `app = Flask(name)`

@app.route decorator	A decorator in Flask used to map URLs to specific functions in a Flask application.	`@app.route('/')`
		`def hello_world():`
		`    return "My first Flask application in action!"`

200 OK status	Flask servers automatically return a 200 OK status when you return from the @app.route method. 200 is also returned by default when you use the jsonify() method to respond to a request. A successful response with a status code of 200 will be sent back when the given code executes.	`@app.route('/')`
		`def hello_world():`
		`    return ("My first Flask application in action!", 200)`

Error 404	**400** indicates an invalid request. This status could imply the parameters are missing or improper or the request is invalid in another way.**401** indicates the credentials are missing or invalid.**403** implies that the client credentials are not sufficient to fulfill the request.**404** If the server is unable to find the resource, it returns a 404 status.**405** indicates that the requested operation is not supported.	`@app.route('/')`
		`def search_response():`
		`    query = request.args.get("q")`
		`    if not query:`
		`        return {"error_message": "Input parameter missing"}, 422`
		`    # fetch the resource from the database`
		`    resource = fetch_from_database(query)`
		`    if resource:`
		`        return {"message": resource}`
		`    else:`
		`        return {"error_message": "Resource not found"}, 404`

Error 500	500 is used when there is an error on the server.	`@app.errorhandler(500)`
		`def server_error(error):`
		`    return {"message": "Something went wrong on the server"}, 500`

Author(s)

Andrew Pfeiffer

Other Contributor(s)

Abhishek Gagneja, Sina Nazeri

Mark as completed

Like Dislike Report an issue